



AIR UNIVERSITY
DEPARTMENT OF MECHATRONICS ENGINEERING
SEMESTER: SPRING 2026
Computer Programming
CE-112

Assignment-2

Instructor: Umer Farooq

Due Date: 07th May 2026

Soft Copy Time: 10:00 PM

Name **Muhammad Mujtaba Bin Zahid**

Name in HEX **4D7568616D6D6164204D756A746162612042696E205A61686964**

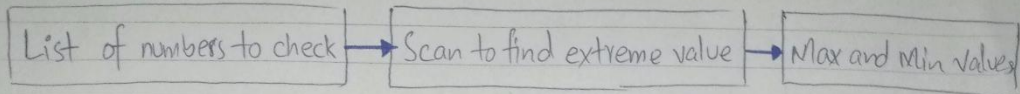
Roll.no **2501538** **Roll.no in 4bit Hex** **262BA2**

Question No	Write Status (Complete/Not complete and issue that you faced) (Compulsory, Question will be checked on bases of this index)	Remarks Must be filled for every question
1	Completed	Accurately finds minimum and maximum array values
2	Completed	Computes standard deviation using the math library
3	Completed	Computes 3x3 matrix multiplication successfully
4	Completed	Successfully parses strings for hidden characters
5	Completed	Calculates mean, median, and mode correctly

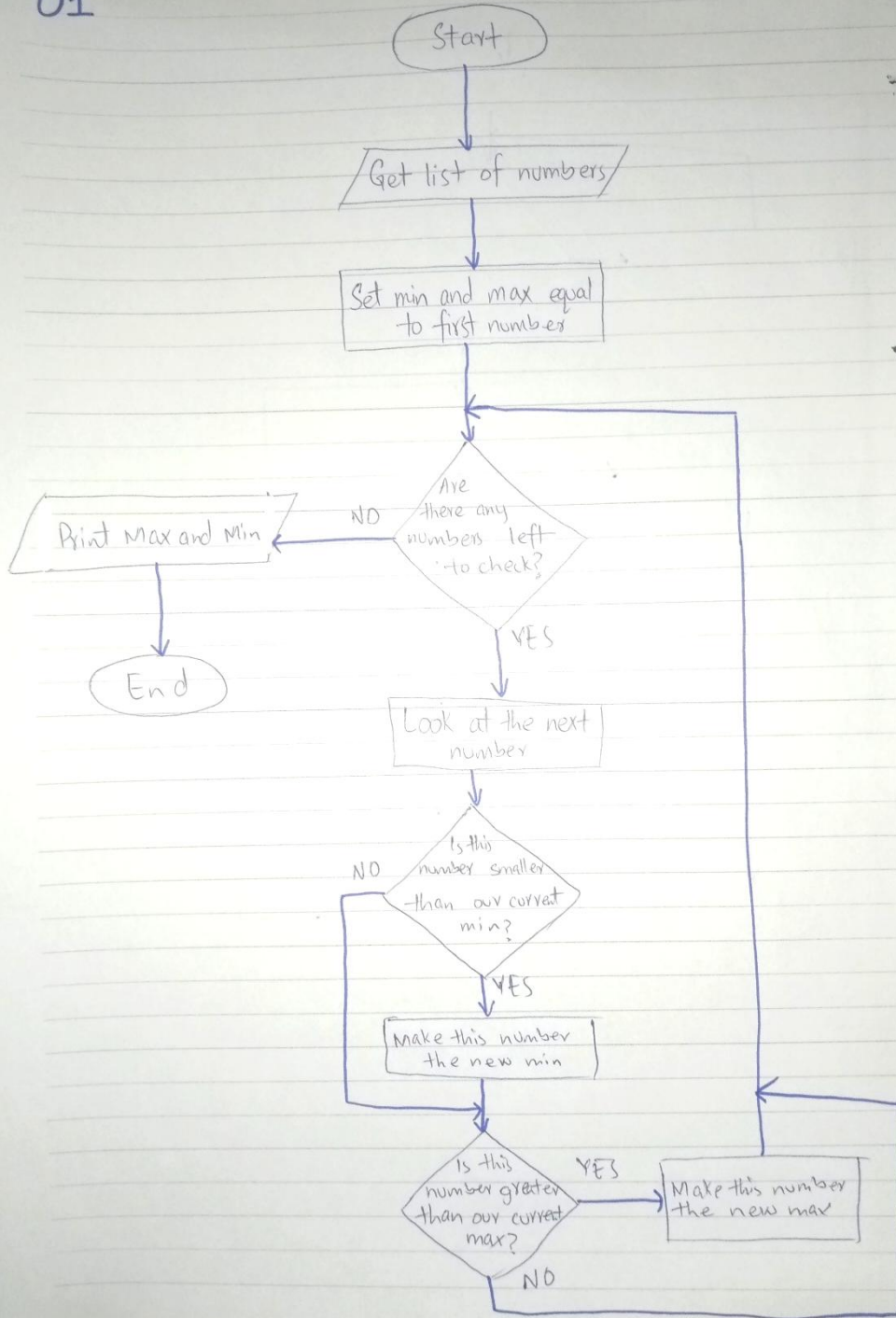
Q1) Write and test the following function that returns through its reference parameters both the maximum and the minimum values stored in an array: void getExtremes(float& min, float& max, float a[], int n);

```
Code 01
1  #include <iostream>
2  #include <stdlib.h>
3  //library for system()
4  using namespace std;
5  void getExtremes(float& min, float& max, float a[], int n);
6  // function declaration and passing variables by reference (&)
7  int main() {
8      float max, min;
9      // variable initialization
10     float arr[] = {67, -15, 0, 1, 100};
11     // array definition
12     int size = sizeof(arr)/sizeof(arr[0]);
13     // calculate the size of array
14     cout << "Array is: ";
15     for (int i = 0; i < size; i++) {
16         cout << arr[i] << " ";
17     }
18     // outputs the array
19     getExtremes(min, max, arr, size);
20     // function is called to get the extremes
21     cout << "\n\nMin: " << min;
22         << "\nMax: " << max;
23     return 0;
24 }
25 void getExtremes(float& min, float& max, float a[], int n) {
26     // function to get the minimum and maximum value from array
27     min = a[0];
28     max = a[0];
29     for (int i = 0; i < n; i++) {
30         if (min > a[i]) { min = a[i]; }
31         if (max < a[i]) { max = a[i]; }
32     } // finds the maximum and minimum
33 }
34 }
```

01



01



Array is : 67 -15 0 1 100

Min: -15

Max: 100

Q2) Write and test the following function:

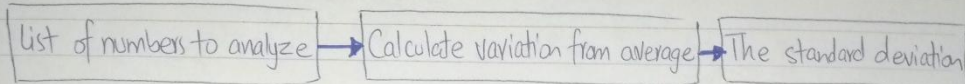
double stdev(double x[], int n);

The function returns the standard deviation of a data set of n numbers $x_0, \dots, x_{n-1}$.

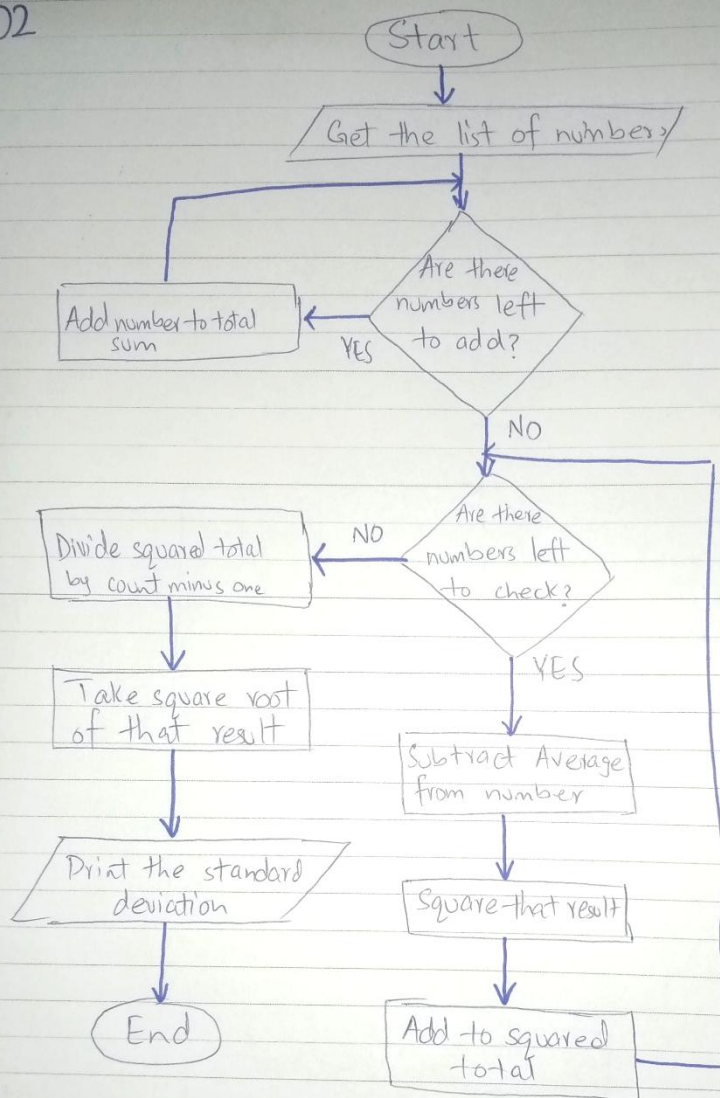
Code 02

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4 double stdev(double x[], int n);
5 // function declaration
6 int main() {
7     double arr[] = {90000, 67.677, 4567, 420, 23000};
8     // array initialization
9     int size = sizeof(arr)/sizeof(arr[0]);
10    // calculate the size of array
11    cout << "The standard deviation is: " << stdev(arr, size);
12    // function is called to calculate the standard deviation
13    return 0;
14 }
15 double stdev(double x[], int n) {
16    // function definition
17    double sum = 0, avg = 0, dev, div, sqsum = 0, diff = 0;
18    // initializing required variables for calculation
19    for (int i = 0; i < n; i++) {
20        // calculates the sum
21        sum += x[i];
22    }
23    avg = sum/n; // calculated the average or x-bar
24    for (int i = 0; i < n; i++) {
25        // loop to calculate the sigma (i=0 to i=n-1)
26        diff = x[i] - avg;
27        sqsum = pow(diff, 2);
28    }
29    div = sqsum/(n-1);
30    dev = sqrt(div);
31    return dev;
32    // returns the standard deviation
33 }
```

02



02



The Standard Deviation is: 38284

Q3) Write a program to multiply two 3x3 matrices. Make with function that can pass array to each function.

GetValue(PassArray)

ComputeAxA(PassArray)

PrintArray(PassArray)

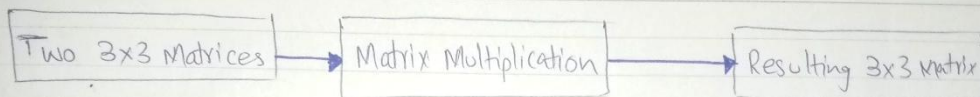
```
Code 03
1 #include <iostream>
2 using namespace std;
3 int rows const rowA = 3, colA = 3, rowB = 3, colB = 3;
4 // set the rows and columns of each matrix using global variables
5 void GetValueA(int A[rowA][colA]);
6 // separate function declaration for each array because order can be different
7 void GetValueB(int B[rowB][colB]);
8 void ComputeAxB(int A[rowA][colA], int B[rowB][colB], int C[rowA][colB]);
9 // function to compute AxB
10 void PrintArray(int A[rowA][colB]);
11 // function to print the resulting matrix
12 int main() {
13     if (colA != rowB) {
14         // a check to validate if multiplication is possible
15         cout << "Multiplication isn't possible.";
16         return 0;
17     }
18     int arr1[rowA][colA], arr2[rowB][colB], result[rowA][colB];
19     // initialization of three arrays
20     cout << "Enter Values for Array 1." << endl;
21     GetValueA(arr1);
22     // getting input for both arrays by calling the specific function
23     cout << "Enter Values for Array 2." << endl;
24     GetValueB(arr2);
25     ComputeAxB(arr1, arr2, result);
26     // function called to compute the result
27     PrintArray(result);
28     // function called to print the resulting matrix
29     return 0;
30 }
31 void GetValueA(int A[rowA][colA]) {
32     // function to get inputs for first matrix
33     for (int i = 0; i < rowA; i++) {
34         for (int j = 0; j < colA; j++) {
35             cout << "[" << i+1 << "]" << j+1 << " = ";
36             cin >> A[i][j];
37         }
38         cout << endl;
39     }
}
```

```

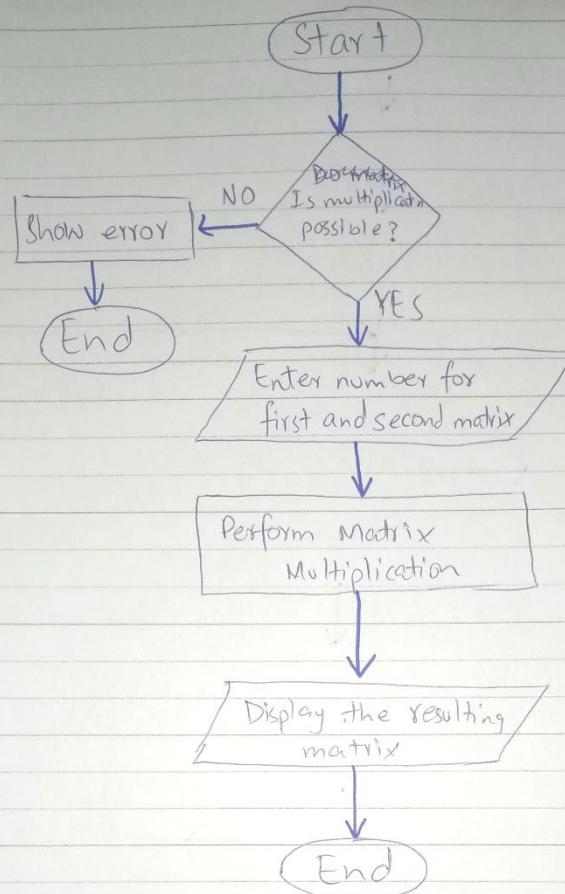
40 }
41 void GetValueB(int B[rowB][colB]) {
42     // function to get input for second matrix
43     for (int i=0; i<rowB; i++) {
44         for (int j=0; j<colB; j++) {
45             cout << "[" << i+1 << "]" << j+1 << " = ";
46             cin >> B[i][j];
47         }
48         cout << endl;
49     }
50 }
51 void ComputeAxB(int A[rowA][colA]; int B[rowB][colB]; int C[rowA][colB]) {
52     // function to multiply the A and B matrices correctly
53     for (int i=0; i<rowA; i++) {
54         for (int j=0; j<colB; j++) {
55             C[i][j] = 0;
56             for (int k=0; k<rowB; k++) {
57                 C[i][j] += A[i][k] * B[k][j];
58             }
59         }
60     }
61 }
62 void PrintArray (int A[rowA][colB]) {
63     // function to print the resulting matrix
64     cout << "\nA x B\n";
65     for (int i=0; i<rowA; i++) {
66         for (int j=0; j<colB; j++) {
67             cout << A[i][j] << "\t";
68         }
69         cout << endl;
70     }
71 }

```

03



03



Enter Values for Array 1.

[1][1] = 11

[1][2] = 12

[1][3] = 13

[2][1] = 21

[2][2] = 22

[2][3] = 23

[3][1] = 31

[3][2] = 32

[3][3] = 33

Enter Values for Array 2.

[1][1] = 41

[1][2] = 42

[1][3] = 43

[2][1] = 1

[2][2] = 52

[2][3] = 53

[3][1] = 61

[3][2] = 62

[3][3] = 63

A x B

1256 1892 1928

2286 3452 3518

3316 5012 5108

Q4) Write a program that takes a string from the user. The program then calculates the

- ✓ Total number of Characters
- ✓ Total number of Spaces
- ✓ Total number of Tabs
- ✓ Total number of Lines

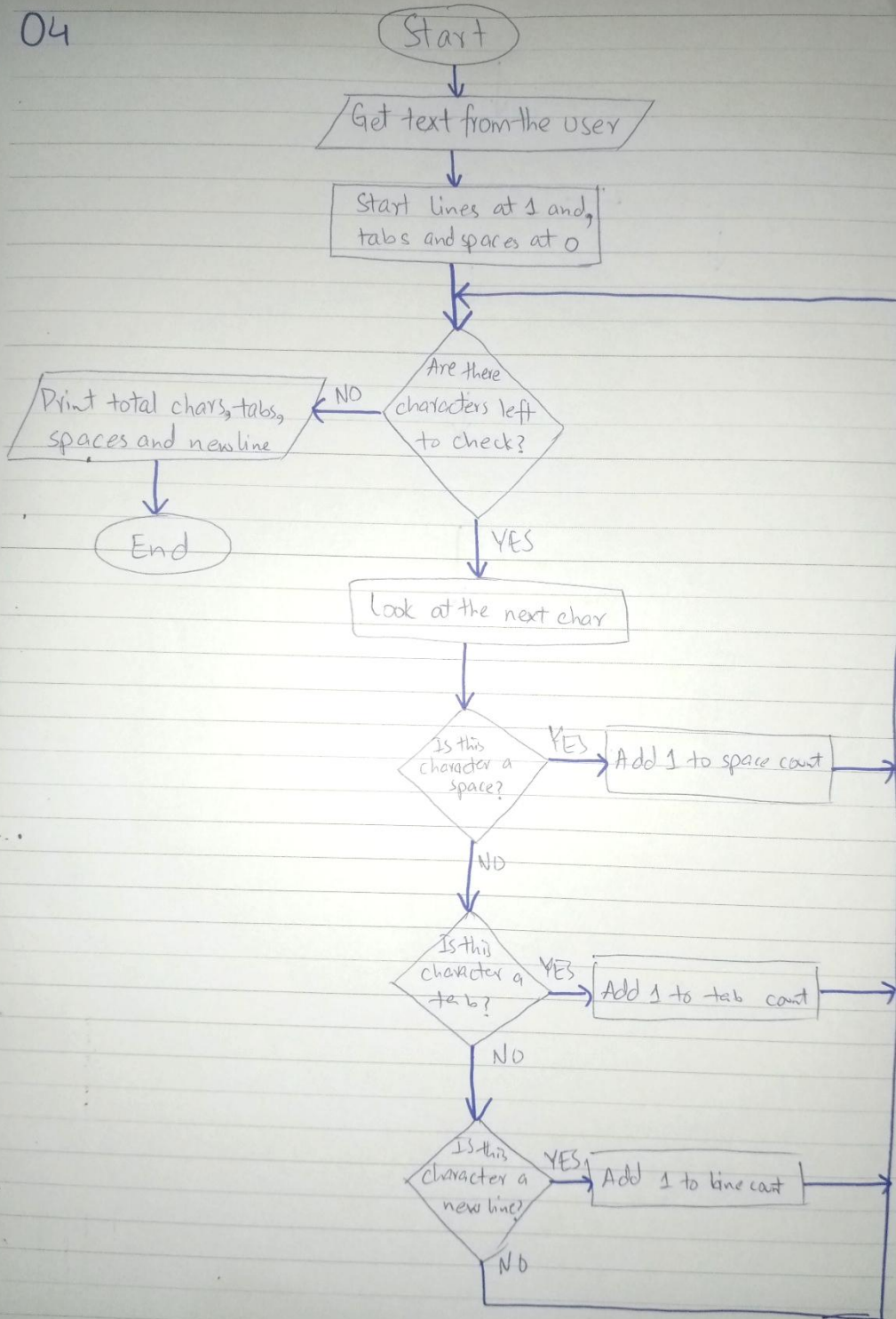
Code 04

```
1 #include <iostream>
2 #include <string>
3 // library for the function .size()
4 using namespace std;
5 void StringEval(int& tabs, int& spaces, int& lines, string sent);
6 // function declaration and passing variables by reference (&)
7 int main() {
8     string sent;
9     int tabs=0, spaces=0, lines=1;
10    // initializing the required variables
11    cout << "Enter a string: (Add '*' at the end of input and press Enter) \n";
12    getline(cin, sent, '*');
13    // getting input with a special marker '*' to indicate the end of input
14    StringEval(tabs, spaces, lines, sent);
15    // calling the function to get the required metrics about the string
16    cout << "\nCharacters : " << sent.size();
17    // total characters can be found by the length of string
18    << "\nSpaces : " << spaces;
19    << "\nTabs : " << tabs;
20    << "\nLines : " << lines;
21    // outputs the 4 metrics
22    return 0;
23 }
24 void StringEval(int& tabs, int& spaces, int& lines, string sent) {
25    // function to calculate tabs, spaces, lines
26    for (int i=0; i<sent.size(); i++) {
27        /* loop to check the string character by character
28        and increasing the respective counter if a match is found */
29        if (sent[i] == ' ') {
30            spaces++;
31        } else if (sent[i] == '\t') {
32            tabs++;
33        } else if (sent[i] == '\n') {
34            lines++;
35        }
36    }
37 }
```

04



04



Enter a string: (Add '*' at the end of input and Press Enter)

Item 1: Apple

Item 2: Banana

Indent test.*

Characters : 42

Spaces : 3

Tabs : 3

Lines : 3

Q5) Write a program to calculate mean, median and mode of an array using function.

Code 05

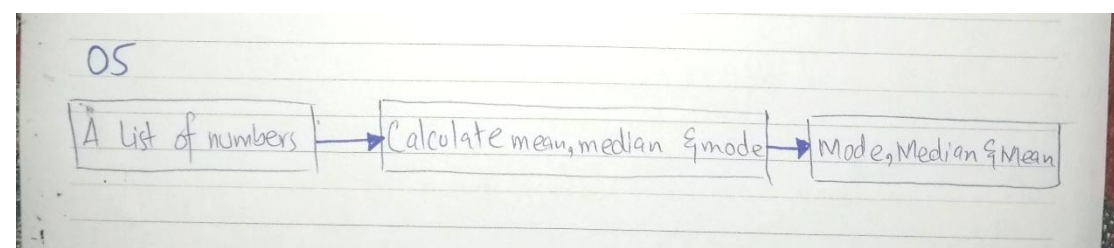
```
1 #include <iostream>
2 #include <algorithm>
3 // Library for the function sort()
4 using namespace std;
5 // function declaration
6 double getMean(int a[], int n);
7 double getMedian(int a[], int n);
8 void getMode(int& mode, int& occurrence, int a[], int n);
9 // passing variables by reference (&)
10 int main() {
11     int arr[] = {34, 56, 34, 23, 13, 45, 34, 78, 55, 99};
12     // array initialization
13     int size = sizeof(arr) / sizeof(arr[0]);
14     // calculating size of array
15     int mode = 0, occurrence = 0;
16     // initializing required variables
17     cout << "Array is : ";
18     for (int i = 0; i < size; i++) {
19         cout << arr[i] << " ";
20     }
21     // outputs the array
22     getMode(mode, occurrence, arr, size);
23     // function called to calculate the mode
24     cout << "\n\nMean : " << getMean(arr, size)
25         // function called to get mean
26         << "\nMedian : " << getMedian(arr, size)
27         // function called to get median
28         << "\nMode : " << mode
29         << "appeared " << occurrence << " times";
30     // output the mode and how many times it appeared
31 }
```



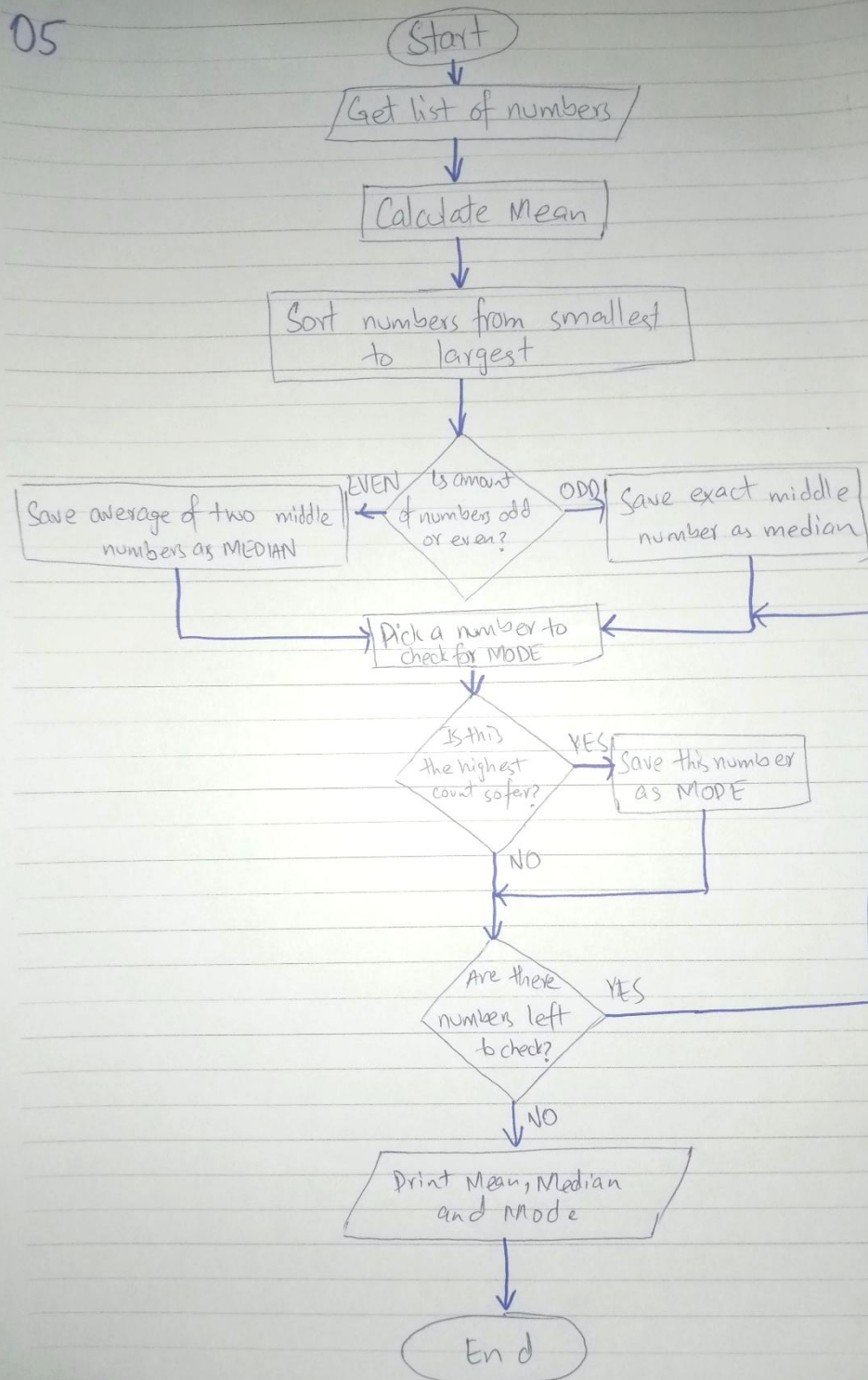
```

32 double getMean(int a[], int n) {
33     // function to calculate the mean
34     double sum = 0;
35     for (int i = 0; i < n; i++) {
36         sum += a[i];
37         // calculate the sum
38     }
39     // return the mean/average
40     return sum/n;
41 }
42 double getMedian(int a[], int n) {
43     // function to calculate the median
44     sort(a, a+n);
45     // sort the array in increasing order
46     if (n % 2 != 0) {
47         return a[n/2];
48         // if size of array is odd, return the middle value
49     } else {
50         return (a[n/2] + a[(n-1)/2]) / 2;
51         /* if size of array is even,
52            return the average of two middle values */
53     }
54 }
55 void getMode(int &mode, int &occurrence, int a[], int n) {
56     // function to calculate the mode and occurrence
57     for (int i = 0; i < n; i++) {
58         // nested loop to match each value with whole array
59         int count = 0;
60         // variable to find occurrence
61         for (int j = 0; j < n; j++) {
62             if (a[i] == a[j]) {
63                 count++;
64                 // increment
65             }
66         }
67         if (count > occurrence) {
68             // finds the value which appeared the most
69             occurrence = count;
70             mode = a[i];
71         }
72     }
73 }

```



05



Array is : 34 56 34 23 13 45 34 78 55 99

Mean : 47.1

Median : 39

Mode : 34 appeared 3 times

